

CAS4160 Homework 6: Model-based RL

[Kim Minsung]

[2020142087]

1 Introduction

The goal of this assignment is to get experience with model-based reinforcement learning. In general, model-based reinforcement learning consists of two main parts: learning a dynamics function to model observed state transitions, and then using predictions from the learned model to decide what to do. Model predictions can be used to directly find an action that maximizes predicted rewards, or to learn a policy.

In this assignment, you will implement learning a dynamics model and planning actions with the learned model. Specifically, you will implement CEM, which generates the action only by planning through the model, without any explicit policy.

2 Code structure overview

The code structure is similar to that of the previous homework. In this assignment, you mainly work with the following codes:

- `cas4160/scripts/run_hw6.py`: the main training loop for your implementation.
- `cas4160/agents/model_based_agent.py`: the learner you will implement.

3 Model-based reinforcement learning

We provide a brief overview of model-based reinforcement learning (MBRL), and the specific type of MBRL you will be implementing in this homework. MBRL primarily consists of two parts: (1) learning a dynamics model (Section 3.1) and (2) using the learned dynamics models to plan and execute actions that maximize a reward function (Section 3.2).

3.1 Learning dynamics model

In this assignment, you will learn a neural network dynamics model f_θ of the form:

$$\delta_{t+1} = f_\theta(\mathbf{s}_t, \mathbf{a}_t), \quad (1)$$

which predicts the change in state given the current state and action. So, given the prediction δ_{t+1} , you can generate the next state prediction with

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \delta_{t+1}. \quad (2)$$

See this [paper](#) for intuition on why we might want our network to predict state differences, instead of directly predicting next state.

You will train f_θ in a standard supervised learning setup, by performing gradient descent on the following objective:

$$L(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}) \in \mathcal{D}} [\|(\mathbf{s}_{t+1} - \mathbf{s}_t) - f_\theta(\mathbf{s}_t, \mathbf{a}_t)\|_2^2]. \quad (3)$$

In practice, it is helpful to normalize the target values of a neural network. Thus, we will train the network to predict a normalized version of the change in state:

$$L(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}) \in \mathcal{D}} [\|\text{Normalize}(\mathbf{s}_{t+1} - \mathbf{s}_t) - f_\theta(\mathbf{s}_t, \mathbf{a}_t)\|_2^2]. \quad (4)$$

Since f_θ is trained to predict the normalized state difference, you can predict the next state with

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \text{Unnormalize}(f_\theta(\mathbf{s}_t, \mathbf{a}_t)). \quad (5)$$

For simplicity, we define $F_\theta(\mathbf{s}, \mathbf{a}) = \mathbf{s} + \text{Unnormalize}(f_\theta(\mathbf{s}, \mathbf{a})) = \mathbf{s}'$.

3.2 Action optimization

Given the learned dynamics model f_θ , we now want to select and execute actions that maximize a known reward function. The optimal actions can be found by solving the following optimization:

$$\mathbf{a}_t = \text{argmax}_{\mathbf{a}_{t:\infty}} \sum_{t'=t}^{\infty} r(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}), \text{ where } \hat{\mathbf{s}}_{t'+1} = F_\theta(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}) \text{ and } \hat{\mathbf{s}}_t = \mathbf{s}_t. \quad (6)$$

However, solving Equation (6) is impractical for two reasons: (1) planning over an infinite sequence of actions is impossible and (2) the learned dynamics model is imperfect, so using it to plan in such an open-loop manner will lead to accumulating errors over time and planning far into the future will become very inaccurate.

Instead, we can solve the following finite-sequence, finite-length optimization problem:

$$A^* = \text{argmax}_{\{A^{(0)}, \dots, A^{(K-1)}\}} \sum_{t'=t}^{t+H-1} r(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}), \text{ where } \hat{\mathbf{s}}_{t'+1} = F_\theta(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}) \text{ and } \hat{\mathbf{s}}_t = \mathbf{s}_t, \quad (7)$$

where $A^{(k)} = (\mathbf{a}_t^{(k)}, \dots, \mathbf{a}_{t+H-1}^{(k)})$ is a random action sequence of length H . What Equation (7) says is to consider K random action sequences of length H , predict the result (i.e. future states) of taking each of these action sequences using the learned dynamics model f_θ , evaluate the reward associated with each candidate action sequence, and select the best action sequence. Note that this approach only plans H steps into the future, which is desirable because it prevents accumulating model error, but is also limited because it may not be sufficient for solving long-horizon tasks.

A better alternative to this random-shooting optimization approach is the cross-entropy method (CEM), which is similar to random-shooting, but with iterative improvement of the distribution of action sequences that are sampled from. We first randomly initialize a set of K action sequences $\{A^{(0)}, \dots, A^{(K-1)}\}$, as in random shooting. Then, we choose the J sequences with the highest predicted sum of discounted rewards as the “elite” action sequences. We then fit a diagonal Gaussian with the same mean and variance as the elite action sequences, and use this as our action sampling distribution for the next iteration. After repeating this process M times, we take the final mean of the Gaussian as the optimized action sequence.

Additionally, since our model is imperfect and things will never go perfectly according to plan, we adopt a model predictive control (MPC) approach, where at every timestep we perform random shooting or CEM to select the best H -step action sequence, but we execute only the first action from that sequence and replan at the next timestep using updated state information. This reduces the effect of compounding errors when using our approximate dynamics model to plan a far into the future.

3.3 On-policy data collection

Although MBRL is in theory off-policy (meaning it can learn from any data), in practice, it will perform poorly if you do not have on-policy data. In other words, if a model is trained on only randomly-collected

data, it will (in most cases) be insufficient to describe the parts of the state space that we may actually care about. We can therefore use on-policy data collection in an iterative algorithm to improve overall task performance. This is summarized as follows:

Algorithm 1 Model-based RL with on-policy data collection

Input: Run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g. random policy) to collect $D = \{\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}\}$

```

1: while not converged do
2:   // Expand dataset using model rollouts
3:   Train  $f_\theta$  using  $\mathcal{D}$  (Equation (4))
4:    $\mathbf{s}_t \leftarrow$  current state
5:   for  $M$  times do
6:     for  $t = 0, \dots, T$  do
7:        $A^* = \pi_{\text{MPC}}(\mathbf{a}_t | \mathbf{s}_t)$ , where  $\pi_{\text{MPC}}$  is obtained from random shooting or CEM
8:        $\mathbf{a}_t \leftarrow$  first action in  $A^*$ 
9:       Get  $\mathbf{s}_{t+1}$  by executing  $\mathbf{a}_t$  in the environment
10:      Add  $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$  to  $\mathcal{D}$ 

```

3.4 Ensembles

A simple and effective way to improve predictions is to use an ensemble of models. The idea is simple: rather than training one network f_θ to make predictions, we train N independently initialized networks: $\{f_\theta^i\}_{i=1}^N$. At test time, for each candidate action sequence, we generate N independent rollouts and average the rewards of these rollouts to choose the best action sequence.

4 Learning dynamics model

In this section, you will train a model using a dataset collected by executing random actions. Specifically, you will train an ensemble of dynamics model using normalization, as mentioned in Section 3.1. A common practice for training the model is also learning the reward of the environment. However, in this homework, you only learn the dynamics and use the true reward function for simplicity.

4.1 Implementation

Let's implement the model training code in `run_hw6.py` and `model_based_agent.py`:

- `cas4160/scripts/run_hw6.py`: implement all TODOs.
- `cas4160/agents/model_based_agent.py`: implement `get_loss()`.

4.2 Experiments

- Train a model on Halfcheetah with the configuration `experiments/mpc/halfcheetah_0_iter.yaml`.

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_0_iter.yaml
```
- This config will only train the ensemble of dynamics models; thus, there is not policy evaluation. For this experiment, the model loss should go below 0.2 by iteration 500.
- Plot the model loss (use `itr_0_loss_curve.png` inside your logging directory, NOT the tensorboard).

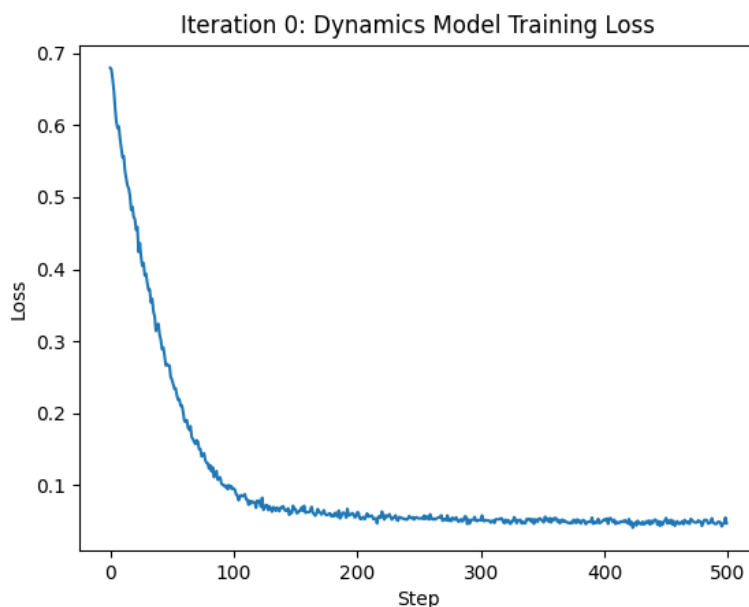


Figure 1: Loss Curve of halfcheetah_0_iter.yaml experiment.

5 Random shooting and CEM

5.1 Implementation

In this section, you will implement MPC with two methods: random shooting and CEM.

- `cs285/agents/model_based_agent.py`: please implement all TODOs.

5.2 Experiments

- Train the dynamics model with on-policy data collected from random shooting with the configuration `halfcheetah_multi_iter.yaml` on Halfcheetah. You would have rewards **300 or higher**. (In fact, if all implementation details are correctly handled, the reward would be 800 or higher. However, we will not deduct points for missing those details.)

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_multi_iter.yaml
```

- Train the dynamics model with on-policy data collected from CEM with the configuration `halfcheetah_cem.yaml` on Halfcheetah. You would have rewards **800 or higher**. (Same as random shooting, if all implementation details are correctly handled, the reward would be 1100 or higher. However, we will not deduct points for missing those details.)

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_cem.yaml
```

- **CEM experiments will take approximately two hours to finish.** If you are using a RTX 3090, we strongly recommend to execute the scripts for the next problem concurrently to maximize the GPU utilization. Otherwise, you would have to wait 6 hours for all experiments.

- Report the results (mean and std. of average returns) for random shooting and CEM. Compare and explain your results.

Answer:

Table 1: Performance with random shooting and CEM.

	Mean	Standard Deviation
Random shooting	847.05	54.77
CEM	2500.27	50.19

Add your explanation.

It was confirmed that CEM improved the average return by nearly three times compared to random shooting. In particular, CEM achieved a very high performance with a mean return of 2500.27, while its standard deviation was 50.19, showing results that were similarly stable or slightly more stable than random shooting (54.77).

These results can be explained by the fact that CEM repeatedly selects elite action sequences and updates their distribution, thereby effectively exploring better action plans. While random shooting relies solely on random sampling, CEM significantly enhances the performance of model predictive control through iterative optimization.

- Train an agent with CEM with the configuration with different hyperparameters on Halfcheetah. Specifically, run the code with $H = 1$ and $K = 50$ by executing the code as below (the original value is $H = 15$, $K = 1000$).

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_cem_H-1.yaml
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_cem_K-50.yaml
```

- Report the CEM results (mean and std. of average returns) with the **3 settings** (original, $H = 1$, $K = 50$). Compare and explain your results.

Answer:

Table 2: Performance of CEM with 3 different hyperparameters.

	Mean	Standard Deviation
CEM	2500.27	50.19
CEM ($H = 1$)	-15250.87	11181.71
CEM ($K = 50$)	1302.09	61.47

Add your explanation.

The original CEM configuration ($H = 15$, $K = 1000$) showed the best performance. When the horizon (H) was significantly reduced to 1, the average return dropped sharply to -15250.87, and the variance (standard deviation) also became extremely large. This occurred because the model was unable to plan sufficiently far into the future, resulting in compounding errors that caused the rewards to become highly negative.

Even when K was reduced to 50, the average return decreased to 1302.09, which is less than half of the performance achieved by the original CEM. This is attributed to the insufficient number of candidate action sequences, which reduced the exploration capability and prevented the model from finding the optimal action sequence.

These results demonstrate that a sufficient planning horizon (H) and a sufficient number of candidate samples (K) are highly important hyperparameters for the performance of model-based reinforcement learning in MPC.

6 Submission

Please submit the code and the “**report**” in a single zip file, `hw6_[YourStudentID].zip`. The zip file should be smaller than 50MB. The structure of the submission file should be:

```
hw6_[YourStudentID].zip
├── hw6_[YourStudentID].pdf
├── cas4160
│   ├── agents
│   │   └── model_based_agent.py
│   └── ...
└── ...
```